# MPMC lock-free queue benchmarking with LTTng tracing

- LTTng tracing evaluation
- MPMC lock-free queues benchmarking

# LTTng

| Feature | LTTng | spdlog | syslog |
|---|---|---|---|
| **Binary Output** | **Yes (CFT)** | **Text** | **Text** |
| **Blocking** | No (ring-buffer) | Yes (can be) | Yes |
| **Kernel tracing** | Yes | No | No |
| **Use Case** | Performance analysis | Application logging | System logging |

# MPMC lock-free Queues

Three queues implemented:

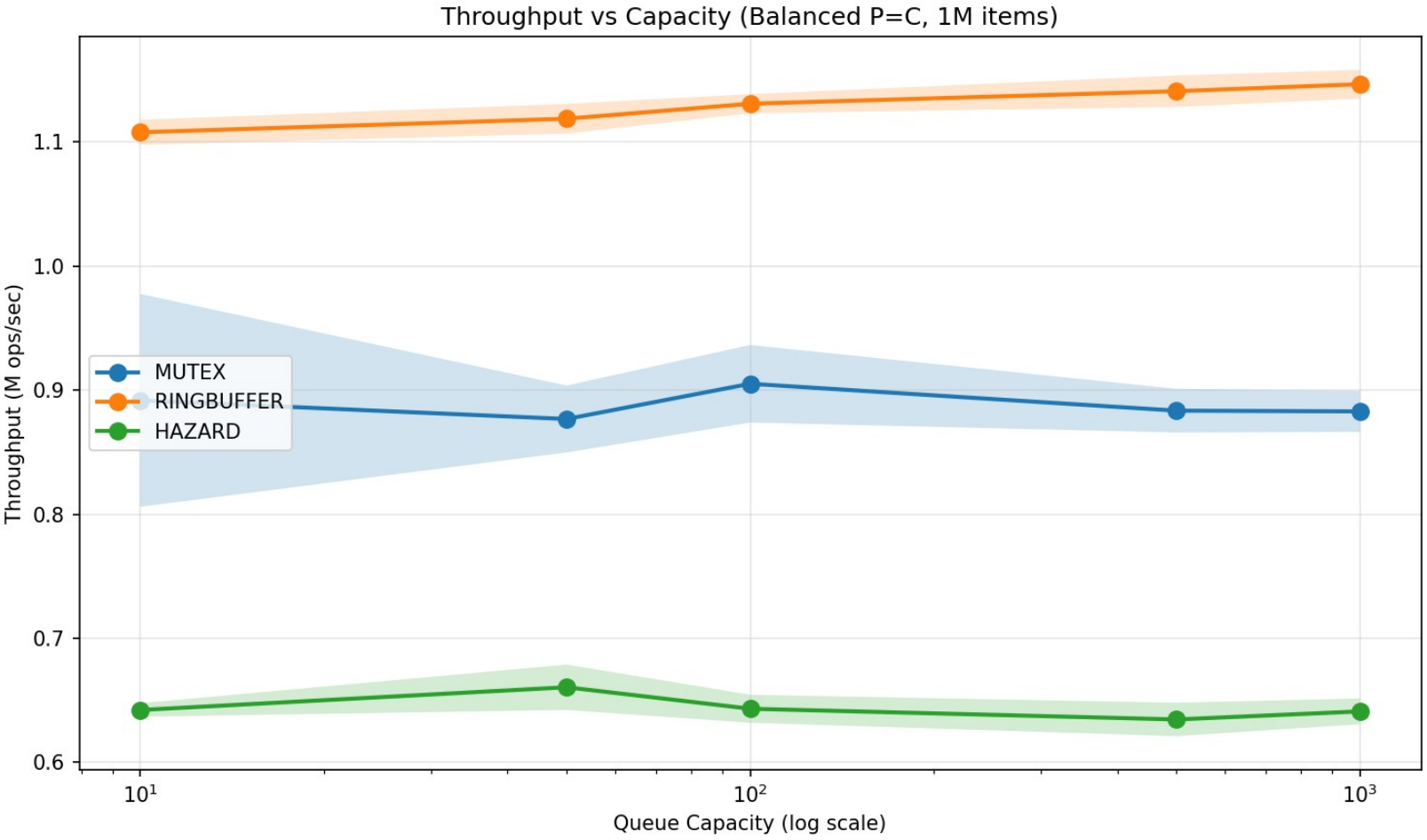
1) MutexQueue - Baseline blocking queue (std::mutex + std::queue)
2) RingBufferQueue - Lock-free bounded queue (CAS on positions)
3) TwoLockHazardQueue - Unbounded queue with hazard pointers (after noticing ABA problem)

All share same API: enqueue(), dequeue(), item_size(), capacity()

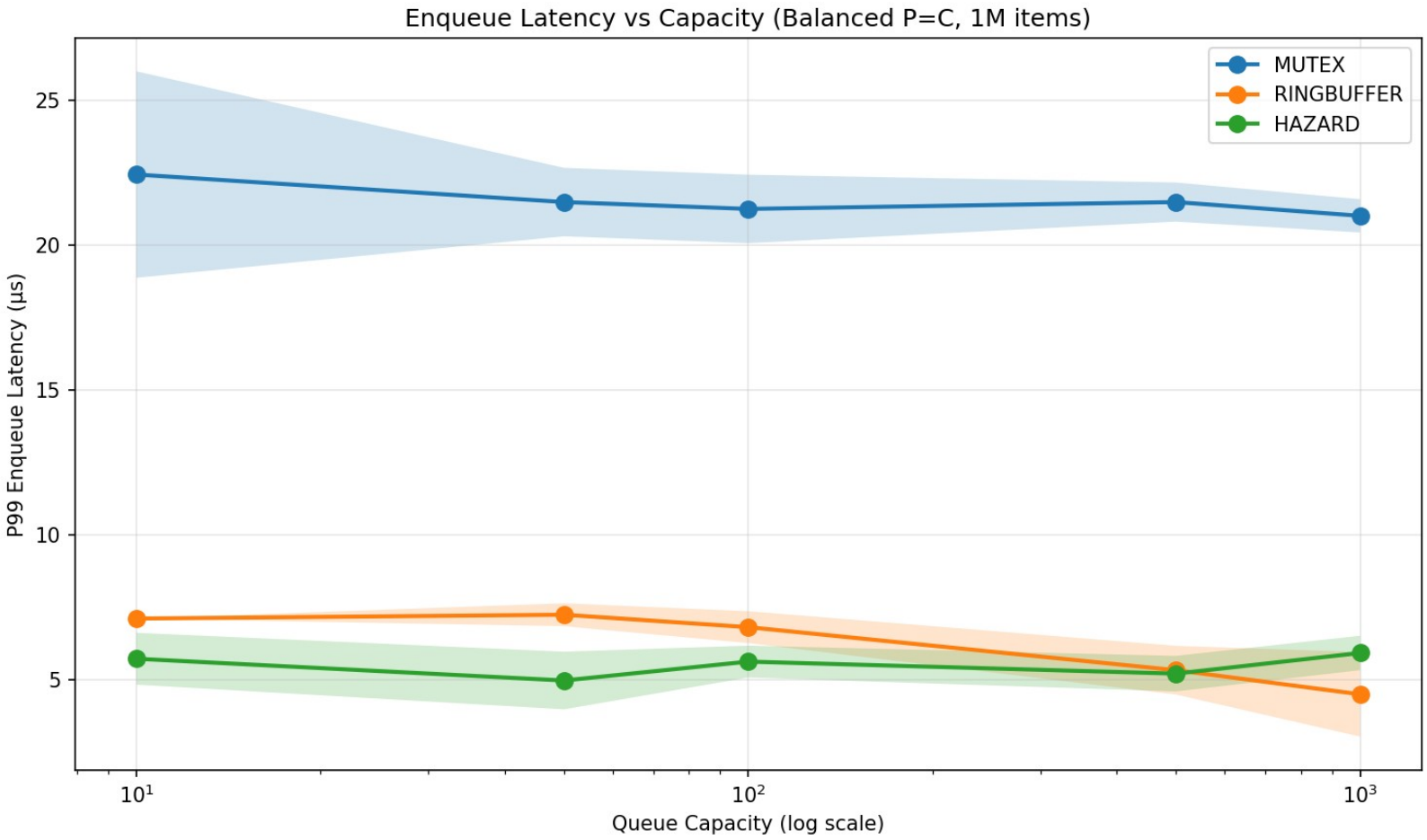
# MPMC lock-free Queues

- Hardware: Ryzen 8845HS laptop (8 cores / 16 threads)
  power-save mode @1400Mhz
- Fixed configuration: 8 producers, 8 consumers, 1M items each
- Queue capacities tested: 10, 50, 100, 500, 1000
- Metrics: Throughput (M ops/sec), P99 latency (μs)
- LTTng trace-points for logging

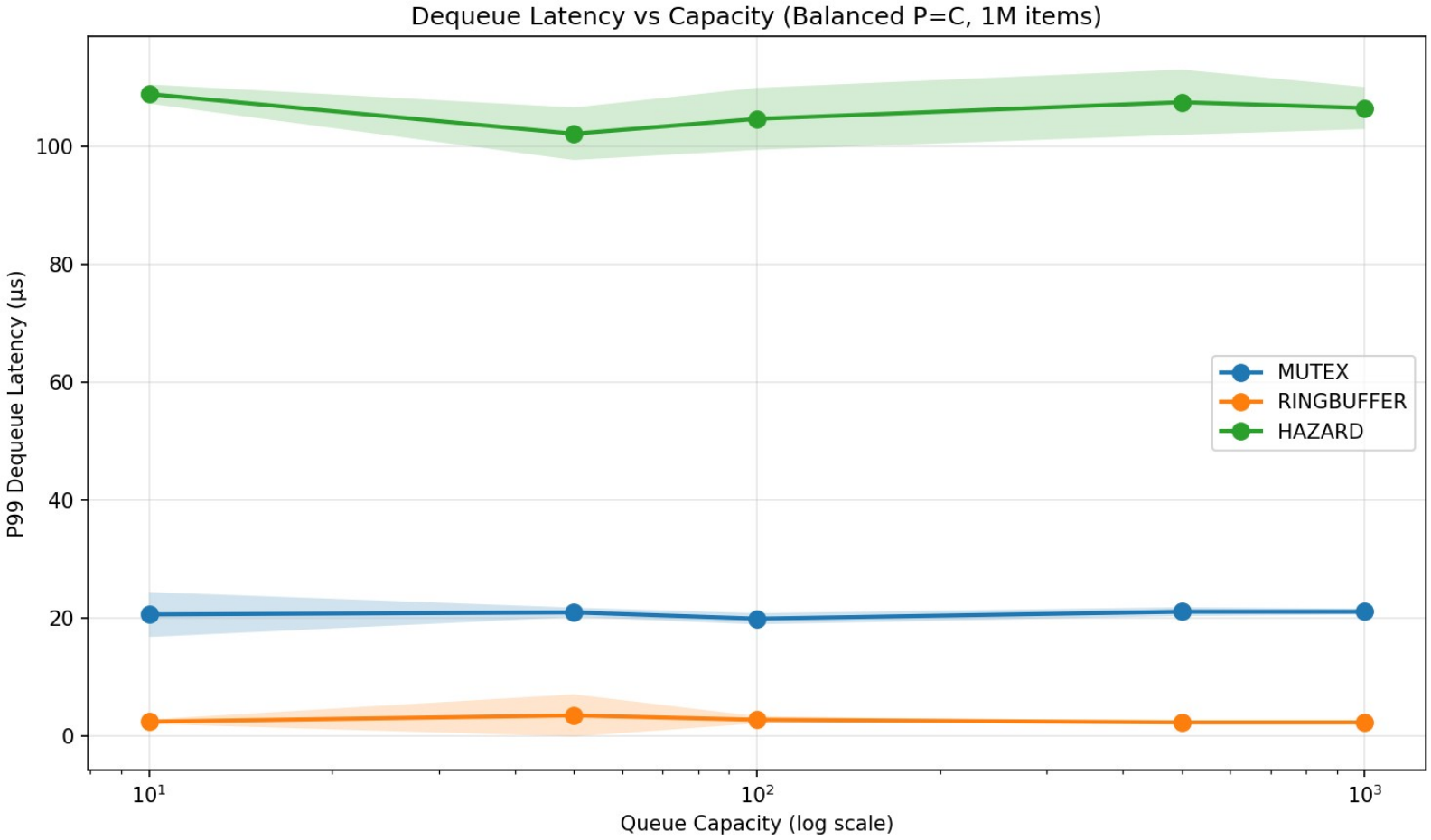
# MPMC lock-free Queues



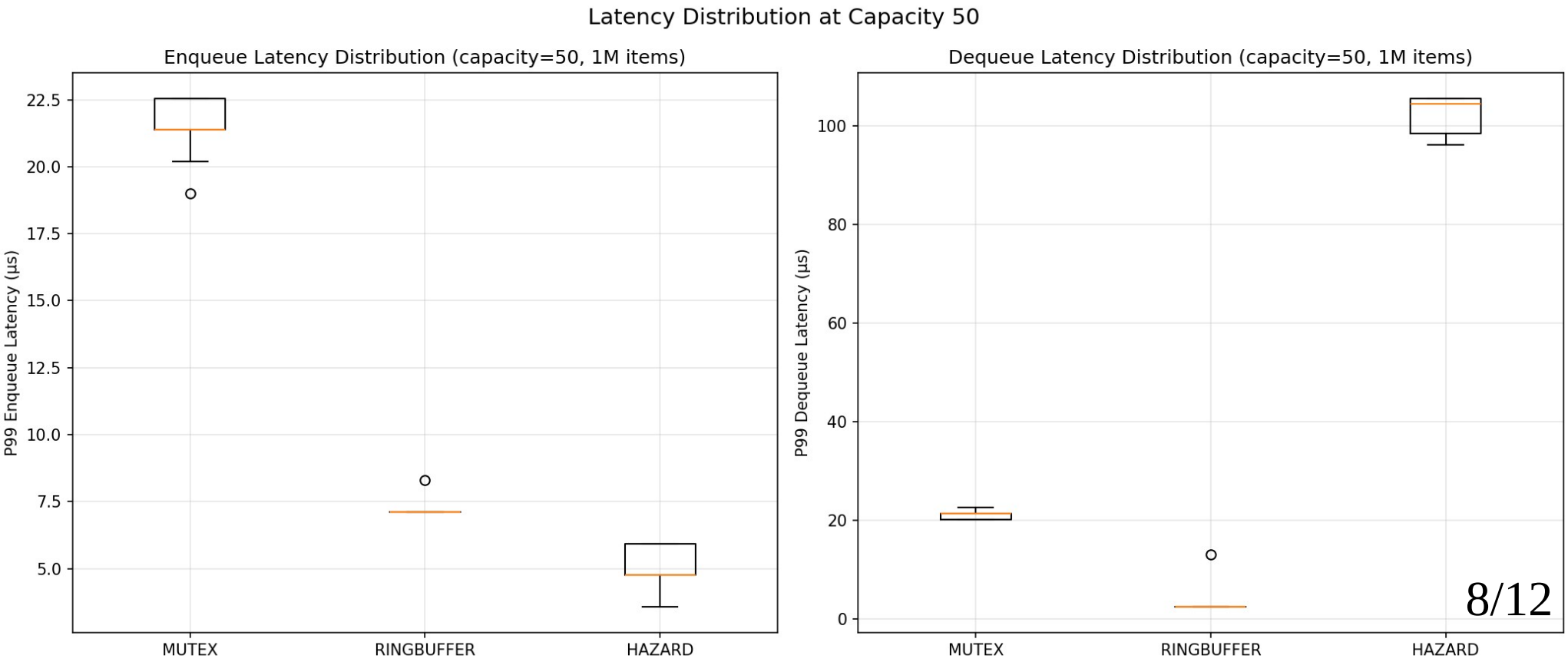
# MPMC lock-free Queues



# MPMC lock-free Queues



# MPMC lock-free Queues



# Overall Conclusions

- Mutex can be competitive regarding throughput thanks to adaptive spinning
- Ring-buffer wins on latency
- Two-lock queue only if unbounded capacity required
- Hazard pointers are correct but expensive
- Bounded queues are simpler and faster than unbounded

# Difficulties during the projcet

- Proper queue cap
- Trace postprocessing due to data size
- Algorithmic scale via runtime limitation over cgroups

# Improvements

- Hazard pointers for Two-lock queue
- LMAX Disruptor
- Better variables for tracepoints

Thank You for your attention!

Q&A